



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table border="1"> <tr> <td>Student 1</td><td>Student 2</td></tr> <tr> <td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr> </table> Name 1: SAFIYA NURSYAHADAH BINTI MASNOOR Metric No: A23CS0176 Link for Video Demo: https://drive.google.com/drive/u/0/folders/1uYREPTtMVFNrgII5N2LoUa1-AOMSahf Name 2: NURUL ASYIKIN BINTI KHAIRUL ANUAR Metric No: A23CS0162 Link for Video Demo: https://drive.google.com/file/d/1AdhKY2hsMWxBFqDxEDFxoQXRY0uzWiWT/view?usp=drive_link	Student 1	Student 2	<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>
Student 1	Student 2				
<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>				
SUBMISSION DATE & ITEMS:	Duration of submission :- 2 weeks Submission items in elearning:- 1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page. 2. The assembly programs (in .asm).				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

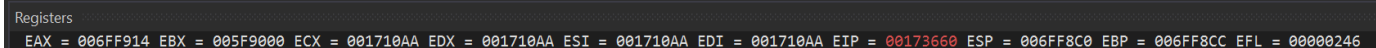
- Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX
LINE3	AX = 0009h BX = 0001h	Switch the value of AX and BX
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX (9d) into var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register BX (1d) into var2

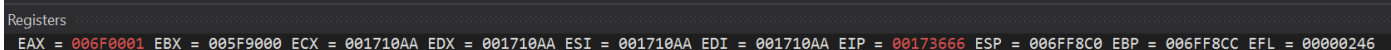
- Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



Registers
EAX = 006FF914 EBX = 005F9000 ECX = 001710AA EDX = 001710AA ESI = 001710AA EDI = 001710AA EIP = 00173660 ESP = 006FF8C0 EBP = 006FF8CC EFL = 00000246

LINE2:



Registers
EAX = 006F0001 EBX = 005F9000 ECX = 001710AA EDX = 001710AA ESI = 001710AA EDI = 001710AA EIP = 00173666 ESP = 006FF8C0 EBP = 006FF8CC EFL = 00000246

LINE3:

Registers

EAX = 006F0001 EBX = 005F0009 ECX = 001710AA EDX = 001710AA ESI = 001710AA EDI = 001710AA EIP = 0017366D ESP = 006FF8C0 EBP = 006FF8CC EFL = 00000246

LINE4:

Registers

EAX = 006F0009 EBX = 005F0001 ECX = 001710AA EDX = 001710AA ESI = 001710AA EDI = 001710AA EIP = 0017366F ESP = 006FF8C0 EBP = 006FF8CC EFL = 00000246

LINE5:

Registers

EAX = 006F0009 EBX = 005F0001 ECX = 001710AA EDX = 001710AA ESI = 001710AA EDI = 001710AA EIP = 00173675 ESP = 006FF8C0 EBP = 006FF8CC EFL = 00000246

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

- Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval ; LINE1
    neg eax      ; LINE2
    mov ebx,Yval ; LINE3
    sub ebx,Zval ; LINE4
    add eax,ebx  ; LINE5
    inc eax      ; LINE6
    mov Rval,eax ; LINE7
    exit
main endp
end main
```

Answer Q2

- Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFE6h	Negate the value of Xval which is 26d to -26d
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFFF6h Zval = 00000028h	Subtract the value in the register EBX (30d) with Zval (40d) and stored the value in EBX
LINE5	EAX = FFFFFFFDCh EBX = FFFFFFFF6h	Add the value of EBX (-10d) and EAX (-26d) and stored the total value in EAX
LINE6	EAX = FFFFFFFDDh	Add 1 to EAX (-36d) and stored the value into EAX
LINE7	EAX = FFFFFFFDDh Rval = FFFFFFFDDh	Move the value in EAX (-35d) into Rval

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 0071FDD0 EBX = 0088E000 ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 00611010 ESP = 0071FD7C EBP = 0071FD88 EFL = 00000246
0x00614004 = 0000001A
```

LINE2:

Registers

EAX = 0000001A EBX = 0088E000 ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 00611015 ESP = 0071FD7C EBP = 0071FD88 EFL = 00000246

LINE3:

Registers

EAX = FFFFFFFE EBX = 0088E000 ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 00611017 ESP = 0071FD7C EBP = 0071FD88 EFL = 00000293

0x00614008 = 0000001E

LINE4:

Registers

EAX = FFFFFFFE EBX = 0000001E ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 0061101D ESP = 0071FD7C EBP = 0071FD88 EFL = 00000293

0x0061400C = 00000028

LINE5:

Registers

EAX = FFFFFFFE EBX = FFFFFFF6 ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 00611023 ESP = 0071FD7C EBP = 0071FD88 EFL = 00000287

LINE6:

Registers

EAX = FFFFFFFDC EBX = FFFFFFF6 ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 00611025 ESP = 0071FD7C EBP = 0071FD88 EFL = 00000283

LINE7:

Registers

EAX = FFFFFFFDD EBX = FFFFFFF6 ECX = 00611005 EDX = 00611005 ESI = 00611005 EDI = 00611005 EIP = 00611026 ESP = 0071FD7C EBP = 0071FD88 EFL = 00000287

0x00614000 = 00000000

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include Irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2             ; LINE2
    add eax, var3        ; LINE3
    dec eax              ; LINE4
    exit
main endp
end main
```

Answer Q3

- Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiply the value of var2 (10d) by EAX (5d) and store it in EAX
LINE3	EAX = 00000046h var3 = 00000014h	Add the value of var3 (20d) into EAX (50d) and store it in EAX
LINE4	EAX = 00000045h var4 = 00000000h	Subtract 1 from register EAX (70d)

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 00CFFB60 EBX = 00A1B000 ECX = 00571005 EDX = 00571005 ESI = 00571005 EDI = 00571005 EIP = 00571010 ESP = 00CFFB0C EBP = 00CFFB18 EFL = 00000246
0x00574000 = 00000005
```

LINE2:

```
Registers
EAX = 00000005 EBX = 00A1B000 ECX = 00571005 EDX = 00571005 ESI = 00571005 EDI = 00571005 EIP = 00571015 ESP = 00CFFB0C EBP = 00CFFB18 EFL = 00000246
0x00574004 = 0000000A
```

LINE3:

Registers

EAX = 00000032 EBX = 00A1B000 ECX = 00571005 EDX = 00000000 ESI = 00571005 EDI = 00571005 EIP = 00571018 ESP = 00CFFB0C EBP = 00CFFB18 EFL = 00000202
0x00574008 = 00000014

LINE4:

Registers

EAX = 00000046 EBX = 00A1B000 ECX = 00571005 EDX = 00000000 ESI = 00571005 EDI = 00571005 EIP = 00571021 ESP = 00CFFB0C EBP = 00CFFB18 EFL = 00000202

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1           ; LINE1
    mov bx,5              ; LINE2
    mul bx                ; LINE3
    mov bx,var2           ; LINE4
    sub bx,3              ; LINE5
    div bx                ; LINE6
    mov var4,ax           ; LINE7
    exit
main endp
end main
```

Answer Q4

- Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move the value 5d into register BX
LINE3	AX = 00C8h BX = 0005h	Multiply the value of BX (5d) with AX (40d) and store the value in AX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2 (10d) into the register BX
LINE5	BX = 0007h	Subtract the value in BX (10d) with 3d
LINE6	AX = 001Ch BX = 0007h DX = 0004h	Divide the value of AX (200d) with the value of BX (7d) and store the value in AX(28d) and DX (4d)
LINE7	AX = 001Ch var4 = 001Ch	Move the value of register AX (28d) into var4

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 00DAFAF4 EBX = 00E8D000 ECX = 007D1005 EDX = 007D1005 ESI = 007D1005 EDI = 007D1005
EIP = 007D1010 ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000246
```


LINE2:

Registers

EAX = 00DA0028 EBX = 00E8D000 ECX = 007D1005 EDX = 007D1005 ESI = 007D1005 EDI = 007D1005
EIP = 007D1016 ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000246

LINE3:

Registers

EAX = 00DA0028 EBX = 00E80005 ECX = 007D1005 EDX = 007D1005 ESI = 007D1005 EDI = 007D1005
EIP = 007D101A ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000246

LINE4:

Registers

EAX = 00DA00C8 EBX = 00E80005 ECX = 007D1005 EDX = 007D0000 ESI = 007D1005 EDI = 007D1005
EIP = 007D101D ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000202

LINE5:

Registers

EAX = 00DA00C8 EBX = 00E8000A ECX = 007D1005 EDX = 007D0000 ESI = 007D1005 EDI = 007D1005
EIP = 007D1024 ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000202

LINE6:

Registers

EAX = 00DA00C8 EBX = 00E80007 ECX = 007D1005 EDX = 007D0000 ESI = 007D1005 EDI = 007D1005
EIP = 007D1028 ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000202

LINE7:

Registers

EAX = 00DA001C EBX = 00E80007 ECX = 007D1005 EDX = 007D0004 ESI = 007D1005 EDI = 007D1005
EIP = 007D102B ESP = 00DAFAA0 EBP = 00DAFAAC EFL = 00000202

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX,0      ; Clear DX
MOV AX,1000h   ; Load 1000h into AX
MOV CX, 25h    ; Load 25h into CX
MUL CX        ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

- a) Screenshot of full program (.asm) :

```
1  INCLUDE Irvine32.inc
2
3  .code
4  main PROC
5  MOV DX, 0; Clear DX
6  MOV AX, 1000h; Load 1000h into AX
7  MOV CX, 25h; Load 25h into CX
8  MUL CX; Multiply AX by CX, storing the result in DX : AX
9  call DumpRegs
10 exit
11
12 main ENDP
13 END main
14
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

Registers

EAX = 01355000 EBX = 0102B000 ECX = 00B90025 EDX = 00B90002 ESI = 00B910AA EDI = 00B910AA EIP = 00B9366F ESP = 0135F938
EBP = 0135F944 EFL = 00000A07

Q6. Given the following instructions as is Code Snippet 2.

- a) Write a full program to execute the Code Snippet 2.
- b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

- a) Screenshot of full program (.asm) :

```
1  include Irvine32.inc
2
3  .code
4  main proc
5      MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
6      MOV AX, 803h; Load the dividend(8003h) into AX
7      MOV BL, 10h; Load the divisor(10h) into BL
8      DIV BL; Divide DX : AX by BL, whereby AX = quotient & DX = remainder
9      call DumpRegs
10     exit
11     main ENDP
12     END main
13
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

Registers

EAX = 008F0380	EBX = 007B3010	ECX = 003210AA	EDX = 00320000	ESI = 003210AA	EDI = 003210AA	EIP = 0032366C	ESP = 008FF820
EBP = 008FF82C	EFL = 00000246						